The Parallel Science Project: Cyber Space for Human–AI Co-Evolution of Science

Boris Bolliet[1]*, Thomas Borrett[1], Andy Nilipour[2], Pablo Villanueva-Domingo[3], Francisco Villaescusa-Nav

[1]University of Cambridge, [2]University of Oxford, [3]University of Valencia, [4]Simons Foun

April 8, 2026

Abstract

We present Parallel Science, an open infrastructure for autonomous scientific discovery in which fleets of AI research agents operate continuously, generating hypotheses, writing and executing code, analyzing data, and producing papers—all under human supervision. The system integrates five principal components: (i) a multi-agent orchestration framework providing planning-and-control execution; (ii) a research backend that decomposes scientific tasks into subtasks delegated to specialized agents; (iii) a fleet management layer that deploys and monitors multiple autonomous scientists as isolated containers; (iv) a messaging gateway enabling human supervisors to interact with agents through standard communication channels; and (v) a paper repository that automatically indexes, versions, and serves published results. Each agent runs a full research pipeline—from exploratory data analysis through iterative hypothesis refinement to LaTeX paper generation and publication—with every step committed to version control. A real-time dashboard provides cost, progress, and status transparency across the fleet. We describe the architecture, deployment model, and the end-to-end data flow from research task to indexed publication. The system is operational and has produced its first papers, demonstrating that continuous, parallel, human-supervised AI research is technically feasible at modest cost.

1 Introduction

Scientific discovery has historically been constrained by human bandwidth: the number of hypotheses that can be conceived, the volume of literature that can be surveyed, and the experiments that can be designed, executed, and analyzed within a researcher's working day. Recent advances in large language models (LLMs) and multi-agent systems have opened the possibility of delegating substantial portions of the research cycle to autonomous agents [Lu et al., 2024, Boiko et al., 2023, Huang et al., 2024].

We introduce **Parallel Science**, an open infrastructure designed around a simple premise: AI agents should operate as *scientists*—not merely as tools invoked on demand, but as persistent entities that carry out full research cycles under human supervision. The system is designed so that multiple such agents can work simultaneously across different research questions, models, and datasets, while human supervisors retain oversight at every stage.

The key design principles are:

1. **End-to-end autonomy with human oversight.** Each agent runs a complete research pipeline—data exploration, idea generation, methodology design, computation, evaluation,

*Corresponding author: `boris.bolliet@parallelscience.org`. Authors listed in alphabetical order.

paper writing, and publication—but must report results and receive permission from a human supervisor before proceeding to the next stage.

2. **Parallelism at the fleet level.** Multiple agents operate concurrently, each in an isolated environment with its own resources, model configuration, and communication channel. This enables exploration of diverse research directions simultaneously.

3. **Reproducibility through version control.** Every step of every research project is committed to a Git repository. The full provenance of any result—from initial data description to final paper—is traceable through the commit history.

4. **Transparency of cost and progress.** A real-time dashboard tracks each agent's status, resource usage, pipeline progress, and cumulative cost, enabling informed allocation decisions.

5. **Open publication.** Completed papers are automatically published to a dedicated repository modeled on the arXiv preprint server, with stable identifiers, versioning, and category classification.

This paper describes the infrastructure that makes this possible. We focus on the system architecture, deployment model, and data flow rather than the research methodology of individual agents, which will be detailed in a companion paper on the Denario research pipeline [Bolliet et al., 2025].

The remainder of the paper is organized as follows. Section 2 describes the overall system architecture and the role of each component. Section 3 details the fleet management and containerization model. Section 4 traces the end-to-end data flow from research task to published paper. Section 5 describes the publication infrastructure. Section 6 presents the monitoring and cost-tracking dashboard. Section 7 covers the deployment model. Section 8 reports on early operational experience. Section 9 discusses limitations and future directions.

2 System Architecture

Parallel Science is composed of five interconnected open-source projects arranged in a dependency chain:

$$\text{AG2} \longrightarrow \text{CMBAgent} \longrightarrow \text{Denario} \longrightarrow \text{Fleet (Docker)} \longrightarrow \text{OpenClaw}$$

with a publication layer (Parallel ArXiv) operating independently. We describe each component's role in the infrastructure; implementation details of the research pipeline itself are deferred to the companion paper.

2.1 Multi-agent orchestration: AG2

At the foundation of the stack is AG2 [AG2 Team, 2024], a multi-agent orchestration framework evolved from Microsoft's AutoGen [Wu et al., 2023]. We maintain a vendored fork published as `cmbagent_autogen`. AG2 provides:

- **Conversable agents** with configurable system prompts, tool registrations, and LLM backends.
- **Group chat patterns** including sequential, swarm, and nested chat orchestration.
- **Multi-provider support** for OpenAI, Anthropic, Google, and other LLM APIs.

AG2 is used directly by CMBAgent for inter-agent communication during the planning and control phases.

2.2 Research backend: CMBAgent

CMBAgent [Bolliet et al., 2024] is the core research backend that implements a **two-phase planning-and-control strategy**:

1. **Planning phase.** A planner agent, guided by a plan reviewer, iterates to produce a structured execution plan. The plan specifies which specialized agents to invoke, in what order, and with what inputs. The plan is stored in a shared context accessible to all agents.

2. **Control phase.** A controller agent executes the plan step by step. Each subtask is delegated to a specialized agent—an engineer for code generation and execution, a researcher for data analysis, a summarizer for synthesis, or domain-specific agents loaded from configuration.

Each agent is defined by a Python implementation paired with a YAML configuration file, making it straightforward to add new specialized agents for different domains. CMBAgent is domain-agnostic by design; domain specificity enters through agent configuration and available data.

2.3 Research pipeline: Denario

Denario [Bolliet et al., 2025] is a scientific research assistant built on top of CMBAgent. It implements the full research workflow—from exploratory data analysis to paper writing—using a dual-backend architecture that selects between CMBAgent (for complex reasoning tasks such as code execution and analysis) and LangGraph [LangChain Team, 2024] (for faster structured generation tasks such as idea and method formulation). The details of Denario's research methodology are the subject of a companion paper; here we note only that it exposes its capabilities through a **Model Context Protocol (MCP) server** [Anthropic, 2024] with 13 tool endpoints that can be invoked by the messaging gateway.

2.4 Messaging gateway: OpenClaw

Each autonomous scientist is fronted by an OpenClaw gateway instance—a multi-channel messaging platform supporting over 20 communication channels including Slack, WhatsApp, Telegram, and Discord. OpenClaw serves two roles in the architecture:

1. **Human–agent interface.** Supervisors interact with their assigned scientists through familiar messaging platforms. The gateway handles message routing, threading, file uploads, and streaming responses.

2. **Tool orchestration.** When a research request arrives, the gateway invokes Denario's MCP tools to trigger pipeline steps. Long-running operations (which may take minutes to hours) are managed asynchronously, with status updates streamed back to the supervisor.

OpenClaw's plugin architecture ensures that adding new communication channels does not require modifications to the core gateway or the research pipeline.

3 Fleet Management

A central design decision in Parallel Science is that each autonomous scientist runs as an **isolated Docker container** with its own resources, model configuration, workspace, and communication channel. The fleet management layer handles container orchestration, configuration generation, and lifecycle management.

3.1 Configuration

The fleet is configured through a single Python file (`config.py`) that serves as the source of truth for:

- The number of scientists (default: 12).
- The default LLM model and per-scientist overrides. The current fleet uses MiniMax-M2.7 [MiniMax, 2025] as the default, with selected scientists running Claude Sonnet 4.6 [Anthropic, 2025].
- Resource allocation per scientist: CPU count, memory limit, and optional GPU assignment.
- Port allocation for gateway and bridge endpoints.
- Per-scientist parameter overrides for the research pipeline (model selection, timeouts, hardware constraints).

A setup script (`setup.py`) reads this configuration and dynamically generates a Docker Compose file, per-scientist configuration directories, and environment variable mappings.

3.2 Container architecture

Each scientist container is built from a multi-stage Dockerfile that produces a runtime image containing:

- A Node.js runtime running the OpenClaw gateway.
- A Python 3.12 virtual environment with the full AG2–CMBAgent–Denario stack installed from source.
- A LaTeX distribution for paper compilation.
- Git and the GitHub CLI for version-controlled publishing.

On startup, an entrypoint script installs system prompt files (defining the agent's identity and operating instructions), patches the MCP server configuration with container-specific API keys, configures Git authentication, and launches the gateway.

3.3 Isolation and shared resources

Each scientist has isolated volumes for configuration, workspace, and work output. A shared read-only data volume provides common datasets, and a shared tools volume provides utility scripts (e.g., the GitHub Pages builder). This design ensures that scientists cannot interfere with each other's state while sharing access to common resources.

Resource limits are enforced through Docker cgroup constraints. The current fleet allocates between 2 and 32 CPUs and between 2 GB and 64 GB of RAM per scientist, with one scientist having access to an NVIDIA RTX PRO 6000 Blackwell GPU (96 GB VRAM) for computationally intensive tasks.

3.4 Agent identity and instructions

Each scientist's behavior is governed by two instruction files installed at startup:

- A **system prompt** (`SOUL.md`) defining the agent's identity as an autonomous research scientist, specifying the research pipeline workflow, reporting requirements, error handling procedures, and the critical rule that no pipeline step may begin without supervisor permission (unless explicitly waived).
- **Standing instructions** (`AGENTS.md`) providing operational guidance: tool usage notes, container resource awareness, Git workflow, cancellation handling, and memory management.

These files are version-controlled and shared across the fleet, ensuring consistent behavior while allowing per-scientist configuration overrides for model selection and resource constraints.

4 Research Pipeline: End-to-End Data Flow

The complete data flow from a research task to a published paper proceeds through the following stages:

1. **Task assignment.** A human supervisor sends a research task to a scientist via a messaging channel (e.g., Slack). The message is routed through the OpenClaw gateway to the agent runtime.
2. **Project setup.** The agent calls `denario_setup`, which initializes a project directory structure, writes a data description, creates a GitHub repository in the `ParallelScience` organization, and pushes an initial commit.
3. **Pipeline execution.** The agent executes the research pipeline step by step: exploratory data analysis, idea generation, methodology design, computational analysis, and evaluation. Each step invokes a Denario MCP tool, which delegates to the appropriate backend (CMBAgent or LangGraph). After each step, the agent uploads the output file to the messaging channel, reports a summary, and waits for supervisor permission to proceed.
4. **Iterative refinement.** The evaluation step assesses the quality of results and may trigger additional iterations. In each iteration, the evaluator generates feedback that informs the next round of methodology and analysis. The system supports configurable maximum iteration counts.

5. **Paper generation.** When the evaluator determines that results are satisfactory (or the maximum iteration count is reached), the agent generates a LaTeX paper, compiles it to PDF, and commits both to the repository.

6. **Classification.** A two-step hierarchical classifier assigns the paper to arXiv categories. First, an LLM selects a top-level archive (e.g., `physics`, `astro-ph`) from 21 standard archives. Then, conditioned on the archive, a second LLM selects a primary subcategory and up to three secondary categories from the full arXiv taxonomy. Classification never falls back to defaults—if the classifier returns an invalid category, it raises an error and the agent must retry.

7. **Publication.** The agent builds a GitHub Pages site from the paper, updates the repository README with metadata, enables GitHub Pages, and pushes the final commit.

8. **Indexing.** A GitHub webhook fires on the `page_build` event, notifying the Parallel ArXiv service. The service scrapes the Pages site, assigns a stable identifier, and indexes the paper in its database. The paper is live within approximately 60–90 seconds of the push.

Every step produces a Git commit with a descriptive message, creating a complete provenance trail. If any Git operation fails, the research pipeline continues—commits accumulate locally and are pushed when connectivity is restored.

5 Publication Infrastructure: Parallel ArXiv

To provide a unified view of all research output, we operate **Parallel ArXiv**, a paper listing service modeled on the arXiv preprint server. It is deployed as a Flask application on Google Cloud Run and serves papers at `papers.parallelscience.org`.

5.1 Identifier scheme

Papers receive identifiers in arXiv format: `PX:YYMM.NNNNN` (e.g., `PX:2604.00001` for the first paper indexed in April 2026). Identifiers are assigned by an append-only registry that maps repository names to PX IDs. Once assigned, an identifier never changes, even if the paper content is updated.

5.2 Version tracking

When a paper's content changes (detected via SHA-256 hash of title, author, abstract, and categories), a new version is created under the same identifier. Previous versions and their PDFs are preserved. The abstract page displays the full submission history with links to each version.

5.3 Database and storage

The paper database is a SQLite file with three tables: an ID registry (append-only), an ID sequence (for $O(1)$ allocation), and a papers table storing every version. The database is stored in Google Cloud Storage and downloaded to the container on cold start. After webhook-triggered writes, it is synced back to GCS. PDFs are stored in the same GCS bucket and served directly.

5.4 Ingestion

Two ingestion pathways ensure completeness:

1. **Real-time webhook.** An organization-level GitHub webhook fires `page_build` events to the Parallel ArXiv endpoint. The handler validates the HMAC signature, scrapes the repository's GitHub Pages site, and upserts the paper. This provides near-real-time indexing.

2. **Daily scrape.** A safety-net cron job scrapes all repositories in the organization, performing idempotent upserts. Content hashing prevents spurious version bumps when pages are rebuilt without content changes.

5.5 Pages and routes

The service provides category listings (new submissions with full abstracts, recent submissions with titles and authors), individual abstract pages with version history, PDF serving, BibTeX generation, archive landing pages, and author pages.

6 Dashboard: Mission Control

A real-time monitoring dashboard ("Mission Control") provides fleet-wide visibility. A collector process polls Docker containers and scans project directories every 10 seconds, aggregating:

- **Container status**: running, idle, busy (detected via MCP log recency and CPU usage).
- **Resource usage**: memory consumption and CPU utilization per scientist.
- **Pipeline progress**: per-project progress through the research pipeline stages (EDA, Idea, Literature, Methods, Results, Paper), displayed as a progress bar.
- **Cost tracking**: per-stage, per-iteration cost aggregated from LLM API call logs. Costs are extracted from three sources: pipeline log files, CMBAgent JSON cost reports, and EDA/analysis comparison reports.
- **Paper inventory**: titles, scientists, dates, and total project costs for completed papers.

The dashboard is served as a static site accessible via Tailscale at `dashboard.parallelscience.org`.

7 Deployment

The system is deployed across two environments:

- **On-premise server (Orion).** The scientist fleet, dashboard, and messaging gateways run on a dedicated server equipped with an AMD Ryzen Threadripper PRO 9995WX processor and an NVIDIA RTX PRO 6000 Blackwell GPU. Public access is provided through Tailscale Funnel for the dashboard and selected scientist UIs.
- **Google Cloud Platform.** The Parallel ArXiv paper listing service and the project landing page are deployed on Cloud Run. Paper PDFs and the database are stored in Google Cloud Storage.

Communication between the fleet and external services uses outbound connections only: Slack Socket Mode (outbound WebSocket), GitHub API calls, and GCS uploads. No inbound ports need to be exposed for the core research pipeline.

8 Early Operational Experience

The system has been operational since April 2026 with a fleet of 12 scientists. We report on the first days of operation.

8.1 Paper production

Within the first three days, the fleet produced its first set of papers on topics including damped harmonic oscillator dynamics and economics. These papers were automatically published to GitHub, indexed on Parallel ArXiv with PX identifiers, and made available with full abstract pages, PDFs, and BibTeX entries.

8.2 Cost

Research costs are dominated by LLM API calls. Representative per-paper costs for early projects were $0.61 (4 iterations on damped oscillator dynamics) and $0.81 (4 iterations on an economics problem), with the paper-writing stage accounting for approximately half of the total cost. These figures reflect the use of cost-efficient models (MiniMax-M2.7) for the bulk of the pipeline.

8.3 Model diversity

The fleet currently uses two model families: MiniMax-M2.7 as the default for most scientists, and Anthropic Claude Sonnet 4.6 for selected scientists requiring stronger reasoning capabilities. The per-scientist model override mechanism allows A/B testing of model performance on identical tasks.

8.4 Human interaction patterns

In the default interactive mode, supervisors approve each pipeline step before the agent proceeds. This has proven effective for quality control while maintaining high throughput—a supervisor can oversee multiple scientists simultaneously, reviewing outputs as they arrive in Slack. A “full pipeline” mode is available for trusted, well-understood tasks, in which the agent runs all steps autonomously with status updates between each.

9 Discussion

9.1 Limitations

The current system has several limitations that inform future development:

- **Data dependency.** The research pipeline requires a structured data description as input. The system cannot yet autonomously acquire or curate datasets from external sources.
- **Domain breadth.** While the architecture is domain-agnostic, operational experience has so far been concentrated in physics and economics. Scaling to experimental sciences, biology, or engineering will require additional specialized agents and tool integrations.

- **Evaluation quality.** The automated evaluator is an LLM-based critic. Its assessments, while useful for guiding iteration, do not substitute for expert peer review. The publication infrastructure currently has no peer review mechanism.
- **Single-server deployment.** The fleet runs on a single server. Horizontal scaling across multiple machines would require distributed container orchestration (e.g., Kubernetes).

9.2 Future directions

Several extensions are planned or under development:

- **Inter-agent collaboration.** Currently, scientists operate independently. Enabling agents to read each other's papers, build on previous results, and propose collaborative projects would create a richer research ecosystem.
- **Autonomous data acquisition.** Connecting agents to public data repositories, APIs, and databases would enable them to identify and retrieve relevant datasets without human intervention.
- **Community peer review.** Integrating a review mechanism—whether human, AI-assisted, or hybrid—into the publication pipeline would improve quality control.
- **Federated deployment.** Supporting multi-site deployments where different institutions contribute scientists to a shared research effort.

10 Conclusion

We have described Parallel Science, an open infrastructure for continuous, parallel, human-supervised AI research. The system demonstrates that it is technically feasible to operate fleets of autonomous scientists that execute full research cycles—from hypothesis to published paper—at modest cost and under human oversight. The key enabler is the integration of multi-agent orchestration, containerized isolation, standard messaging interfaces, and automated publication into a coherent pipeline where every step is version-controlled and transparent.

The infrastructure is open source and operational. Papers produced by the system are publicly available at `papers.parallelscience.org`. We hope that Parallel Science contributes to a future in which human and artificial scientists work in parallel, each amplifying the other's capabilities in the pursuit of scientific understanding.

Acknowledgments

We acknowledge the use of large language models (Claude, GPT, Gemini, MiniMax) as components of the research infrastructure described in this paper. The Parallel Science project is built on open-source software including AG2, LangGraph, Flask, and OpenClaw. We thank the developers of these projects.

References

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

Daniil A Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.

Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MLAgentBench: Evaluating language agents on machine learning experimentation. *arXiv preprint arXiv:2310.03302*, 2024.

Boris Bolliet, Pablo Villanueva-Domingo, Iñigo Zubeldia, Licong Xu, Thomas Borrett, Francisco Villaescusa-Navarro, and Andy Nilipour. Denario: An open multi-agent research pipeline. *arXiv preprint arXiv:2510.26887*, 2025.

AG2 Team. AG2: A new open-source framework for multi-agent applications built on AutoGen. 2024. https://github.com/ag2ai/ag2.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Boris Bolliet, Iñigo Zubeldia, Licong Xu, Thomas Borrett, Francisco Villaescusa-Navarro, Andy Nilipour, and Pablo Villanueva-Domingo. CMBAgent: Towards an autonomous agent for cosmological data analysis. *arXiv preprint arXiv:2412.13966*, 2024.

LangChain Team. LangGraph: Build stateful multi-agent applications, 2024. https://github.com/langchain-ai/langgraph.

Anthropic. Model context protocol, 2024. https://modelcontextprotocol.io.

MiniMax. MiniMax-M2.7, 2025. https://www.minimax.io.

Anthropic. Claude 4 model family, 2025. https://www.anthropic.com.